

uni-app骨架屏与触底加载load-more样式设计与实现

在uni-app列表类页面中，骨架屏用于优化数据加载等待体验，触底加载的load-more样式则能清晰反馈“加载中”“无更多”等状态。本文将从样式设计原则、核心实现代码到多端适配细节，完整呈现两者的美观化、实用化实现方案，确保样式与业务场景高度契合。

一、设计核心原则

无论是骨架屏还是load-more样式，都需遵循“视觉统一、反馈清晰、性能轻量”的原则：

- 视觉统一**：骨架屏结构需与真实列表项布局一致，颜色与页面主色调协调；load-more样式风格（字体、颜色、图标）与页面整体设计统一。
- 反馈清晰**：load-more需区分“加载中”“无更多”“加载失败”三种核心状态，通过图标+文字组合让用户快速感知；骨架屏通过渐变动画传递“正在加载”的动态感。
- 性能轻量**：骨架屏避免使用复杂嵌套和图片，优先用纯CSS实现；load-more样式简化DOM结构，确保触底时渲染流畅不卡顿。

二、骨架屏样式设计与实现

骨架屏核心是模拟列表项的“占位结构”，配合渐变动画模拟加载动态效果，常见于商品列表、资讯列表等场景。

1. 核心样式逻辑

- 结构模拟**：复制真实列表项的DOM结构，用纯色块替代文字和图片位置。
- 动态效果**：通过CSS渐变动画（linear-gradient）实现“流光”效果，增强加载感知。
- 适配处理**：使用rpx单位确保多端（手机、平板、小程序）布局一致，避免固定像素导致的变形。

2. 完整代码实现（商品列表骨架屏）

Code block

```
1 <template>
2   <view class="goods-page">
3
4     <view class="page-title">热门商品</view>
5
6
7     <view class="skeleton-container" v-if="loading && goodsList.length === 0">
```

```

8      <view class="skeleton-item" v-for="(item, index) in 5" :key="index">
9
10     <view class="skeleton-img"></view>
11
12     <view class="skeleton-info">
13       <view class="skeleton-title"></view>
14       <view class="skeleton-desc"></view>
15       <view class="skeleton-price"></view>
16     </view>
17   </view>
18 </view>
19
20
21 <view class="goods-list" v-else>
22   <view class="goods-item" v-for="(item, index) in goodsList" :key="index">
23     <image class="goods-img" :src="item.imgUrl" mode="aspectFill"></image>
24     <view class="goods-info">
25       <view class="goods-name">{{ item.name }}</view>
26       <view class="goods-desc">{{ item.desc }}</view>
27       <view class="goods-price">¥{{ item.price }}</view>
28     </view>
29   </view>
30 </view>
31
32
33 <view class="load-more" v-if="goodsList.length > 0">
34   <template v-if="loadingMore">
35     <image class="load-icon" src="/static/icons/loading.png"
mode="widthFix"></image>
36     <text class="load-text">加载中...</text>
37   </template>
38   <template v-else-if="hasMore">
39     <text class="load-text" @click="loadMoreData">点击加载更多</text>
40   </template>
41   <template v-else>
42     <text class="load-text disable">已加载全部数据</text>
43   </template>
44 </view>
45 </view>
46 </template>

```

3. 骨架屏核心样式 (CSS)

通过背景渐变和动画实现“流光闪烁”效果，模拟数据加载中的动态感，同时确保结构与真实列表项完全匹配。

```
1 <style scoped>
2   /* 页面基础样式 */
3   .goods-page {
4     padding: 20rpx;
5     background-color: #fafafa;
6   }
7   .page-title {
8     font-size: 36rpx;
9     font-weight: bold;
10    margin-bottom: 20rpx;
11    color: #333;
12  }
13
14  /* 骨架屏样式 */
15  .skeleton-container {
16    width: 100%;
17  }
18  .skeleton-item {
19    display: flex;
20    align-items: flex-start;
21    padding: 20rpx;
22    background-color: #fff;
23    border-radius: 16rpx;
24    margin-bottom: 20rpx;
25    /* 骨架屏动画容器 */
26    overflow: hidden;
27    position: relative;
28  }
29  /* 骨架屏流光动画 */
30  .skeleton-item::after {
31    content: "";
32    position: absolute;
33    top: 0;
34    left: -100%;
35    width: 50%;
36    height: 100%;
37    background: linear-gradient(to right, transparent, rgba(255,255,255,0.3),
transparent);
38    animation: skeleton-loading 1.5s infinite;
39  }
40  @keyframes skeleton-loading {
41    0% {
42      left: -100%;
43    }
44    100% {
45      left: 150%;
46    }
47  }
```

```

47 }
48 .skeleton-img {
49     width: 160rpx;
50     height: 160rpx;
51     border-radius: 12rpx;
52     background-color: #eee;
53     margin-right: 20rpx;
54 }
55 .skeleton-info {
56     flex: 1;
57     display: flex;
58     flex-direction: column;
59     justify-content: center;
60 }
61 .skeleton-title {
62     width: 80%;
63     height: 36rpx;
64     background-color: #eee;
65     border-radius: 8rpx;
66     margin-bottom: 16rpx;
67 }
68 .skeleton-desc {
69     width: 100%;
70     height: 28rpx;
71     background-color: #eee;
72     border-radius: 8rpx;
73     margin-bottom: 24rpx;
74 }
75 .skeleton-price {
76     width: 30%;
77     height: 32rpx;
78     background-color: #eee;
79     border-radius: 8rpx;
80 }
81
82 /* 真实列表样式（与骨架屏结构对应） */
83 .goods-list {
84     width: 100%;
85 }
86 .goods-item {
87     display: flex;

```

三、触底加载load-more样式设计与实现

load-more样式需覆盖“加载中”“可加载”“无更多”三种核心状态，同时支持点击重试等扩展场景，样式设计需兼顾功能性和美观性。

1. load-more核心样式 (CSS)

Code block

```
1  /* 触底加载样式 */
2  .load-more {
3      width: 100%;
4      padding: 30rpx 0;
5      display: flex;
6      align-items: center;
7      justify-content: center;
8      flex-direction: row;
9  }
10 .load-icon {
11     width: 32rpx;
12     height: 32rpx;
13     margin-right: 16rpx;
14     /* 加载图标旋转动画 */
15     animation: load-rotate 1.2s linear infinite;
16 }
17 @keyframes load-rotate {
18     0% {
19         transform: rotate(0deg);
20     }
21     100% {
22         transform: rotate(360deg);
23     }
24 }
25 .load-text {
26     font-size: 24rpx;
27     color: #666;
28 }
29 /* 可点击加载更多样式 (hover效果) */
30 .load-text:not(.disable) {
31     color: #2c83ff;
32     cursor: pointer;
33 }
34 .load-text:not(.disable):active {
35     opacity: 0.8;
36 }
37 /* 无更多数据样式 */
38 .load-text.disable {
39     color: #999;
40 }
41
42 /* 加载失败样式 (扩展) */
43 .load-fail {
```

```
44     color: #ff4400;
45   }
46   .load-fail:active {
47     opacity: 0.8;
48   }
```

2. 状态切换逻辑（JS）

通过数据状态控制不同样式的展示，结合触底事件和手动点击实现加载逻辑，同时确保无效请求被拦截（延续前文核心逻辑）。

Code block

```
1  <script>
2  import { getGoodsList } from '@api/goods.js';
3
4  export default {
5    data() {
6      return {
7        goodsList: [], // 商品列表数据
8        pageNum: 1, // 当前页码
9        pageSize: 10, // 每页条数
10       loading: true, // 页面初始加载状态（控制骨架屏）
11       loadingMore: false, // 触底加载中状态
12       hasMore: true, // 是否有更多数据
13       categoryId: 1 // 商品分类ID（固定参数）
14     };
15   },
16   onLoad() {
17     // 页面初始化加载数据（显示骨架屏）
18     this.initGoodsList();
19   },
20   onReachBottom() {
21     // 触底时自动加载更多（排除初始加载和手动加载状态）
22     if (!this.loading && !this.loadingMore && this.hasMore) {
23       this.loadMoreData();
24     }
25   },
26   methods: {
27     // 初始化商品列表（显示骨架屏）
28     async initGoodsList() {
29       try {
30         const res = await getGoodsList({
31           pageNum: this.pageNum,
32           pageSize: this.pageSize,
33           categoryId: this.categoryId
34         });
```

```
35     if (res.code === 200) {
36         this.goodsList = res.data.list;
37         // 判断是否有更多数据
38         this.hasMore = this.pageNum * this.pageSize < res.data.total;
39     } else {
40         uni.showToast({ title: res.msg || '加载失败', icon: 'none' });
41     }
42 } catch (error) {
43     console.error('初始化商品列表失败: ', error);
44     uni.showToast({ title: '网络异常', icon: 'none' });
45 } finally {
46     // 关闭骨架屏
47     this.loading = false;
48 }
49 },
50 // 加载更多数据 (控制load-more样式)
51 async loadMoreData() {
52     // 防止重复请求
53     if (this.loadingMore) return;
54
55     try {
56         this.loadingMore = true; // 切换为加载中状态 (显示加载图标)
57         this.pageNum += 1;
58
59         const res = await getGoodsList({
60             pageNum: this.pageNum,
61             pageSize: this.pageSize,
62             categoryId: this.categoryId
63         });
64
65         if (res.code === 200) {
66             // 拼接列表数据
67             this.goodsList = [...this.goodsList, ...res.data.list];
68             // 更新是否有更多数据状态
69             this.hasMore = this.pageNum * this.pageSize < res.data.total;
70         } else {
71             this.pageNum -= 1; // 页码回退
72             uni.showToast({ title: res.msg || '加载更多失败', icon: 'none' });
73         }
74     } catch (error) {
75         this.pageNum -= 1; // 页码回退
76         console.error('加载更多失败: ', error);
77         // 切换为加载失败样式 (可点击重试)
78         this.loadingMore = false;
79         uni.showToast({ title: '网络异常, 点击重试', icon: 'none' });
80     } finally {
81         // 关闭加载中状态 (切换为对应样式)
```

```
82         if (this.hasMore) {  
83             this.loadingMore = false;  
84         }  
85     }  
86 }  
87 }  
88 };
```

四、多端适配与优化细节

1. 多端适配要点

- **单位统一**：所有样式尺寸均使用rpx单位，确保在不同屏幕尺寸的手机、平板上自适应。
- **图标适配**：加载图标使用本地图片（如PNG格式），避免使用网络图片导致加载延迟；小程序端注意图片路径需放在static目录下。
- **动画兼容性**：使用CSS3标准动画（linear-gradient、transform），uni-app会自动转换为各端兼容的语法，无需额外处理。
- **H5端滚动适配**：H5端触底事件可能受滚动容器影响，可在pages.json中配置"enablePullDownRefresh": true确保事件正常触发。

2. 性能优化技巧

1. **骨架屏数量控制**：骨架屏默认展示3-5条即可，过多会增加DOM渲染压力，可通过v-for的length控制。
2. **动画优化**：骨架屏和加载图标的动画使用transform和opacity属性，避免触发浏览器重排（reflow），提升渲染性能。
3. **条件渲染优化**：通过v-if/v-else控制骨架屏和真实列表的显示，确保两者不同时渲染，减少DOM节点数量。
4. **图片懒加载**：真实列表的图片添加lazy-load属性，配合骨架屏进一步提升页面加载速度：

```
<image class="goods-img" :src="item.imageUrl" mode="aspectFill"  
lazy-load></image>
```

3. 交互体验优化

- **加载失败反馈**：加载失败时切换为“点击重试”样式，并通过toast提示用户，提升容错性。
- **骨架屏过渡自然**：骨架屏消失和真实列表出现无生硬切换，通过vue的条件渲染自动实现平滑过渡。

- **load-more点击区域扩大**：可给load-more容器添加padding扩大点击区域，提升移动端点击体验：

```
.load-more {  
  padding: 30rpx 40rpx; /* 左右增加padding扩大点击区域 */  
}
```

五、扩展场景：个性化样式定制

可根据业务场景定制不同风格的样式，以下为常见扩展方向：

1. **卡片式骨架屏**：若列表为卡片布局，可给骨架屏添加阴影效果，增强立体感：

```
.skeleton-item {  
  box-shadow: 0 4rpx 12rpx rgba(0,0,0,0.05);  
}
```

2. **彩色骨架屏**：根据商品类型设置不同颜色的骨架屏（如生鲜类用浅绿色，数码类用浅灰色），提升视觉辨识度：

```
/* 生鲜类商品骨架屏 */  
.skeleton-img {  
  background-color: #e8f5e9;  
}  
.skeleton-title, .skeleton-desc {  
  background-color: #f1f8e9;  
}
```

3. **极简load-more样式**：仅用文字反馈状态，适合简约风格的页面：

```
<view class="load-more simple">  
  <text class="load-text" :class="{loading: loadingMore, disable:  
!hasMore}">  
    {{ loadingMore ? '加载中...' : (hasMore ? '上拉加载更多' : '已到底  
部') }}  
  </text>  
</view>
```

六、总结

骨架屏和触底加载load-more的样式实现，核心是“结构模拟+状态反馈+多端适配”。骨架屏通过与真实列表一致的结构和渐变动画，缓解用户等待焦虑；load-more通过清晰的状态区分和交互反馈，提升加载体验。结合本文的样式代码和逻辑控制，可快速在uni-app项目中落地，同时通过多端适配和性能优化技巧，确保在各平台都能呈现流畅、美观的效果。实际开发中可根据产品设计风格，灵活调整颜色、尺寸和动画参数，实现个性化定制。

（注：文档部分内容可能由 AI 生成）